

AxiSim Software Manual

GPU-accelerated axisymmetric CFD, from sketch to solution

Version 1.04-alpha

Lui Tsumura

June 21, 2026

Abstract

AxiSim is an interactive finite-volume solver for incompressible, axisymmetric (r - z) flow. It combines geometry sketching, structured and unstructured meshing, a GPU (CUDA) SIMPLE solver, and real-time post-processing in a single application. This manual documents how to use the software, what it contains, the algorithms and data structures behind it, and its present limitations. It supersedes the v1.01 manual and reflects the v1.04-alpha codebase.

Contents

1	Introduction	3
2	Disclaimers and Project Status	3
2.1	Development status	3
2.2	Verification and validation	3
2.3	Intended use	4
2.4	Licensing	4
3	System Requirements and Installation	4
3.1	Requirements	4
3.2	Building from source	4
3.3	Runtime files	4
4	Libraries and Dependencies	4
5	What Is Included: Application Overview	5
6	Getting Started: A Typical Workflow	5
6.1	Sketching the geometry	5
6.2	Generating the mesh	6
6.3	Defining boundaries and running	6
7	Mesh Generation	6
7.1	Structured mesh	6
7.2	Unstructured mesh	7
7.3	Finite-volume data structure	7
7.4	Boundary groups	8
7.5	Mesh quality: non-orthogonality	8

8	Solver	9
8.1	Governing equations	9
8.2	The SIMPLE algorithm	9
8.3	Discretisation schemes	9
8.4	Linear solvers	9
8.5	Boundary conditions	9
8.6	Steady vs. transient, convergence, residuals	10
8.7	GPU acceleration and device data	10
9	Post-processing and Results	11
9.1	3D results view	11
9.2	Data picker (3D)	11
9.3	2D inspector and per-cell mesh data	11
9.4	Colorbar and colormaps	12
10	Interactive Features and Camera	12
10.1	3D arcball camera	12
10.2	2D camera	12
10.3	Mouse picking	13
11	Console	13
12	Data Structures Reference	13
13	Save and Load	14
14	Limitations	14
Appendix A: Default Parameters		14

1 Introduction

AxiSim is a research/learning CFD application that solves the incompressible Navier–Stokes equations in axisymmetric coordinates using the finite-volume method (FVM) and the SIMPLE pressure–velocity coupling scheme. The numerically intensive work—assembly of discretised equations and the linear solves—runs on an NVIDIA GPU through CUDA, while the entire workflow (geometry, meshing, solving, and visualisation) lives in one real-time interface built on OpenGL and Dear ImGui.

The application is organised as a pipeline of stages, each with its own GUI tab, sharing a common project state:

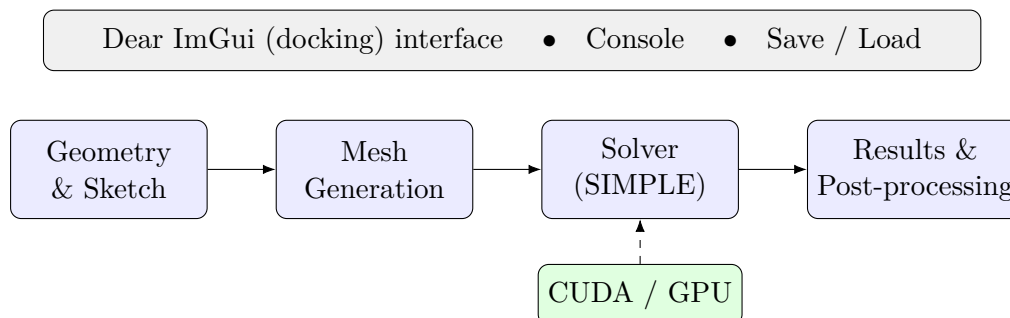


Figure 1: High-level architecture. A shared project flows left to right through four stages; the GUI and console wrap all stages, and the solver offloads to the GPU.

This manual documents the most notable features. Because AxiSim is under active development, some smaller features may be described only briefly, and details are subject to change between versions.

2 Disclaimers and Project Status

AxiSim is alpha-stage, work-in-progress software. Its numerical results have not been formally verified or validated. Do not use it for engineering decisions, safety-critical work, or any application where correctness matters.

2.1 Development status

AxiSim is at version 1.04-alpha. It is updated frequently and changes rapidly; APIs, file formats, default parameters, and behaviour may change without notice between versions. Expect rough edges, incomplete features, and the occasional bug.

2.2 Verification and validation

The solver has *not* undergone systematic verification (e.g. method-of-manufactured- solutions, grid-convergence studies) or validation against analytical solutions, trusted reference codes, or experimental data. Results should be treated as qualitative and exploratory. Known numerical caveats are collected in Section 14.

2.3 Intended use

AxiSim is intended for learning, prototyping, and experimentation with FVM/CFD and GPU programming. It is not a substitute for a validated commercial or open-source solver.

2.4 Licensing

The original AxiSim source code is released under the MIT License. However, AxiSim links the **Gmsh** library, which is licensed under the GNU GPL v2-or-later. *Binaries that are built with and distributed alongside Gmsh are therefore subject to the GPL*, in addition to the MIT terms covering AxiSim’s own source. The third-party components and their licenses are listed in Section 4.

3 System Requirements and Installation

3.1 Requirements

- **Operating system:** Windows 10 / 11 (x64).
- **GPU:** an NVIDIA GPU with CUDA compute capability ≥ 7.5 (Turing / GTX 16 / RTX 20 series and newer) plus a recent driver. AxiSim runs its solver on the GPU and *will not run* on machines without a compatible NVIDIA card.
- **To build from source:** CUDA Toolkit 13.0, a C++20 compiler (Visual Studio 2022), CMake ≥ 3.24 , and vcpkg.

3.2 Building from source

Dependencies that come from vcpkg (GLFW, GLAD, GLM, OpenGL) are installed first; Gmsh and Dear ImGui/ImPlot are bundled in the repository.

```
vcpkg install glfw3 glad glm
cmake --preset x64-Release
cmake --build out/build/x64-Release
```

3.3 Runtime files

The executable loads several resources by relative path, so the following must sit next to **AxiSim.exe** when distributing or running outside the build tree: **glfw3.dll**, **gmsh-4.15.dll**, the **assets/** folder (fonts and icons), and the **graphics/shaders/** folder. The CUDA runtime is statically linked, so end users do not need the CUDA Toolkit—only an NVIDIA driver. The Visual C++ runtime (**MSVCP140.dll**, **VCRUNTIME140*.dll**) must be present (install the Microsoft Visual C++ Redistributable x64 if needed).

4 Libraries and Dependencies

Table 1 lists the notable external components used by AxiSim. See Section 2 for the licensing consequence of linking Gmsh.

Table 1: External libraries and their roles.

Library	Role in Axisim	License
OpenGL	Graphics API for creating the window and rendering models	system
GLSL	Shading language used with OpenGL to render onto the screen	(OpenGL)
GLFW	Window creation, OS context, and input handling	zlib/libpng
GLAD	OpenGL function loader	MIT/public
GLM	Vector / matrix / quaternion math (camera, transforms)	MIT
Dear ImGui (docking)	Main GUI library; docking branch for dockable windows	MIT
ImPlot	Real-time plotting (residual plots)	MIT
Gmsh	Unstructured (triangular) mesh generation	GPL v2+
stb_image	Loading PNG icon/texture assets	MIT/public
NVIDIA CUDA Toolkit	GPU compute: assembly and linear solvers	NVIDIA EULA

5 What Is Included: Application Overview

Axisim ships as a single application whose major components are:

- **Geometry / Sketch view** — draw the axisymmetric cross-section (lines, rectangles, circles, arcs) with live dimensions and snapping.
- **Mesh Inspector** — convert the sketch into a structured (Cartesian) or unstructured (Gmsh) mesh, define boundary groups, add local refinement regions, and inspect per-cell mesh data.
- **Solver** — configure and run the SIMPLE solver; choose discretisation schemes, relaxation, steady/transient mode, and optional scalar transport.
- **Results** — 3D revolved field visualisation and a 2D cross-section inspector with colormaps and a colorbar.
- **Console** — text commands to drive major operations and settings.
- **Residual plot** — live convergence monitoring via ImPlot.
- **Save / Load** — persist and reload project state.

A typical end-to-end session follows the workflow in Figure 2.

6 Getting Started: A Typical Workflow

6.1 Sketching the geometry

The geometry view lets you draw the cross-section of the axisymmetric domain in the (z, r) half-plane. Supported primitives are lines, rectangles, circles, and arcs, with live dimensioning and snapping to existing geometry, the axes, and the origin. A 2D camera (pan with the middle mouse button, zoom with the wheel) frames the sketch.

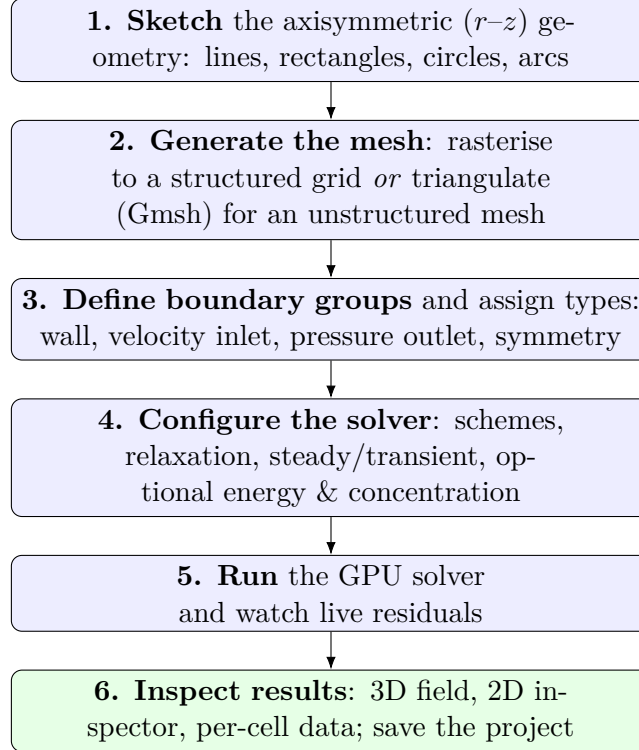


Figure 2: The typical AxiSim workflow.

6.2 Generating the mesh

With a closed sketch, the Mesh Inspector converts the geometry into a finite-volume mesh (Section 7). Choose either a structured Cartesian mesh or an unstructured triangular mesh. Boundary segments are detected automatically and can be grouped and named for boundary-condition assignment.

6.3 Defining boundaries and running

Assign a boundary type to each named group, configure the solver (Section 8), and run. The residual plot updates live so you can monitor convergence. When the run finishes, switch to the Results tab to explore the field.

7 Mesh Generation

AxiSim supports two mesh families, both reduced to a common finite-volume representation (Section 7.3).

7.1 Structured mesh

The structured mesh is a Cartesian grid in the (z, r) plane defined by the number of cells n_z and n_r , the domain length L and radius R , and optional axial/ radial grading (**zBias**, **rBias**). The sketch is *rasterised* onto this grid: every cell whose centre lies outside the sketched domain (or inside an obstacle) is flagged solid. This is recorded in **activeCell** (1 = fluid, 0 = solid) and the

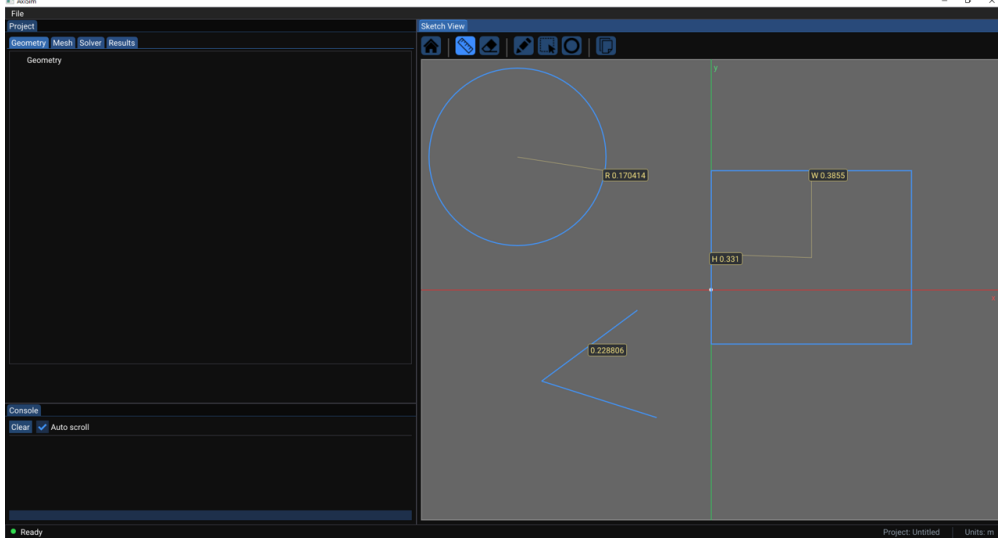
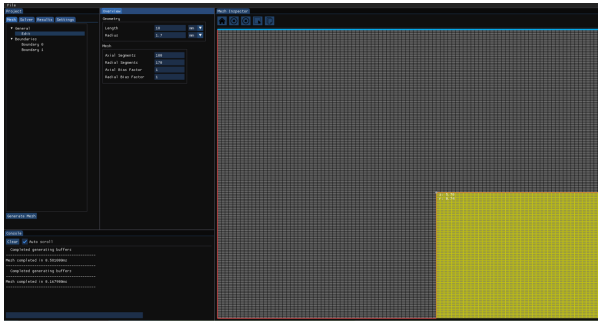


Figure 3: Sketching the axisymmetric cross-section.

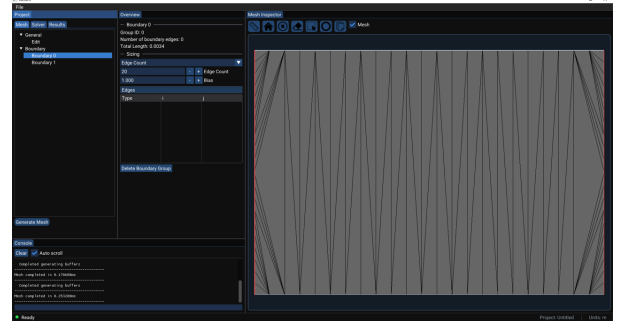
set `obstacleIndices`. Cell index ordering is row-major, $n = i n_z + j$, where i is the radial index and j the axial index.

7.2 Unstructured mesh

The unstructured mesh is a constrained Delaunay triangulation produced by Gmsh from the sketch boundary. Local refinement is controlled by *regions of influence* (rectangular or circular), which set a target element size inside the region and a transition to the background size outside it. Boundary sizing (edge count / target spacing / bias) can be set per boundary group. Each triangle becomes one finite-volume cell.



(a) Structured (Cartesian) mesh.



(b) Unstructured (Gmsh) mesh.

Figure 4: The two meshing modes.

7.3 Finite-volume data structure

Regardless of mesh family, the solver consumes a single `FVMesh` made of cells and faces. A cell stores its centroid, 2D area, revolved volume $V = 2\pi r_c A_{2D}$ (axisymmetric), and the IDs of its bounding faces. A face stores its owner and neighbour cell, its outward unit normal, its centre, and its (revolved) area; a face with `neighbor = -1` is a boundary face.

```

struct FVCell {
    Vec2 center;           // centroid (z, r)
    double area2D = 0.0;   // 2D cross-sectional area
    double volume = 0.0;   // revolved volume 2*pi*r_c*area2D
    std::vector<int> faceIDs;
    bool active = true;    // false => solid / excluded
    bool solid = false;
};

struct FVFace {
    int owner = -1, neighbor = -1; // neighbor < 0 => boundary face
    int v0 = -1, v1 = -1;         // edge endpoints
    Vec2 normal, center;          // outward unit normal; face midpoint
    double length2D = 0.0, area = 0.0;
    int boundaryGroupID = -1;
};

```

Figure 5 shows two adjacent (triangular) cells, the owner P and neighbour N , the centroid-to-centroid vector $\mathbf{d}$, and the face outward normal $\mathbf{S}$. These two quantities drive both the diffusion discretisation and the mesh-quality measure below.

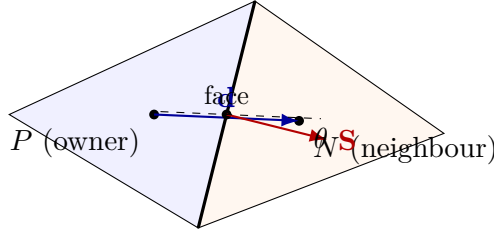


Figure 5: Two adjacent finite-volume cells. $\mathbf{d}$ joins the cell centroids and $\mathbf{S}$ is the shared-face outward normal. The angle θ between them is the *non-orthogonality* (zero for a perfectly orthogonal mesh).

7.4 Boundary groups

Boundary faces are organised into named *boundary groups*. Domain edges and obstacle edges are detected, merged into segments, and the user groups and names them. Each group carries a boundary type and per-field values that the solver reads when applying boundary conditions (Section 8.5).

7.5 Mesh quality: non-orthogonality

For each interior face, the non-orthogonality angle is

$$\theta = \arccos\left(\frac{\mathbf{d} \cdot \mathbf{S}}{|\mathbf{d}| |\mathbf{S}|}\right), \quad (1)$$

where $\mathbf{d}$ is the owner-to-neighbour centroid vector and $\mathbf{S}$ the face normal. A value of 0° means the centroid line is parallel to the face normal (ideal); larger angles increase discretisation error and can slow or destabilise the solve. The Mesh Inspector's cell-pick tool reports each cell's centroid, area/volume, face count, per-face neighbour and length, and the per-cell maximum and average

non-orthogonality. Structured cells report $\approx 0^\circ$ (a useful sanity check), while skewed triangles report larger values.

8 Solver

8.1 Governing equations

AxiSim solves the steady or unsteady incompressible Navier–Stokes equations in axisymmetric (z, r) coordinates (no swirl). Continuity and momentum are

$$\frac{\partial u_z}{\partial z} + \frac{1}{r} \frac{\partial(r u_r)}{\partial r} = 0, \quad (2)$$

$$\rho \frac{Du_z}{Dt} = -\frac{\partial p}{\partial z} + \mu \left[\frac{\partial^2 u_z}{\partial z^2} + \frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial u_z}{\partial r} \right) \right], \quad (3)$$

$$\rho \frac{Du_r}{Dt} = -\frac{\partial p}{\partial r} + \mu \left[\frac{\partial^2 u_r}{\partial z^2} + \frac{\partial}{\partial r} \left(\frac{1}{r} \frac{\partial(r u_r)}{\partial r} \right) \right]. \quad (4)$$

Optional passive scalars—temperature T (energy) and concentration C —are transported by convection–diffusion equations and can be enabled independently (`solveEnergy`, `solveConcentration`).

8.2 The SIMPLE algorithm

Pressure–velocity coupling uses SIMPLE (Semi-Implicit Method for Pressure-Linked Equations). Each outer iteration solves under-relaxed momentum predictors, then a pressure-correction equation, then corrects velocity, pressure, and the face mass flux (Figure 6). Under-relaxation is required for stability; the defaults are momentum ≈ 0.7 and pressure ≈ 0.3 .

8.3 Discretisation schemes

Equations are discretised with the finite-volume method. The available *convection* schemes are first-order upwind, central differencing, second-order upwind, and QUICK. Diffusion uses the centroid-distance gradient along $\mathbf{d}$ plus an optional *non-orthogonal correction* for the cross term (Section 7.5); the number of deferred corrector passes is `nNonOrthCorrectors` (default 0, i.e. orthogonal-only, which is the stable default). Cell gradients (used for the pressure / pressure-correction terms) use either Green–Gauss or least-squares (`GRAD_GREEN_GAUSS`, `GRAD_LSQ`).

8.4 Linear solvers

The discretised systems are solved on the GPU. Selectable iterative solvers include Jacobi and red–black Gauss–Seidel; additional Krylov methods (BiCGStab, GMRES) and a geometric multi-grid path exist in the codebase. Inner/outer iteration counts and tolerances are configurable (`ConfigSimple`, `LinearSolverConfig`, `IterationConfig`).

8.5 Boundary conditions

Boundary conditions are applied per boundary group. The supported boundary *types* are wall, velocity inlet, pressure outlet, and symmetry (Table 2); internally each field at each group is realised as a Dirichlet or Neumann condition with a per-group value. Axisymmetry implies a symmetry condition on the centreline.

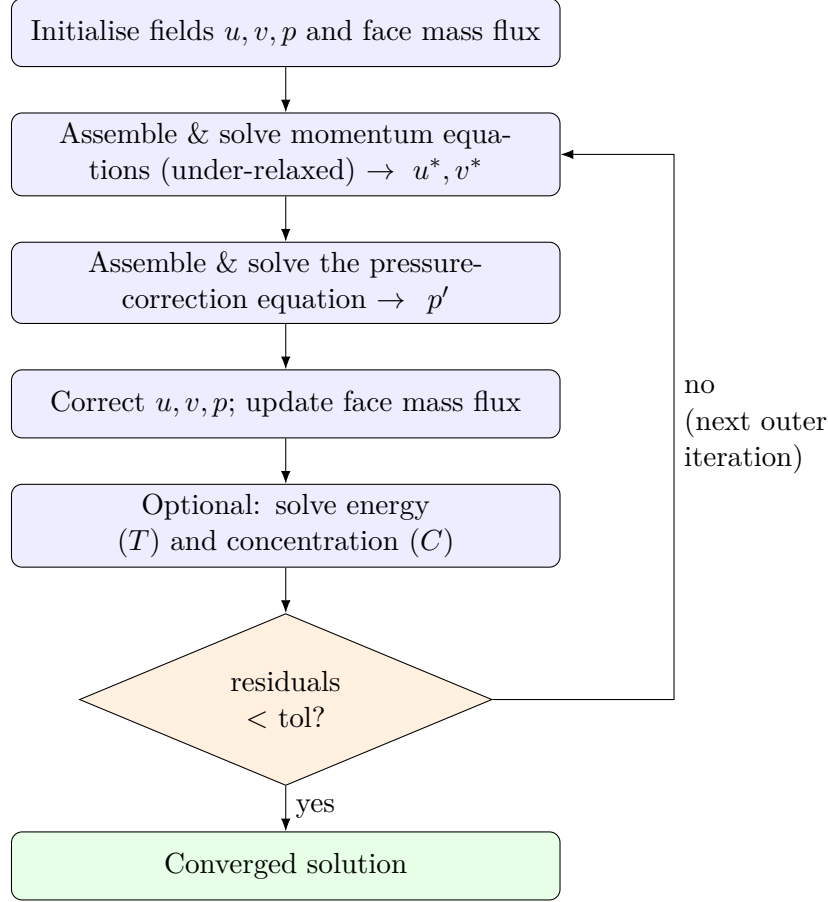


Figure 6: One outer iteration of the SIMPLE loop as implemented in AxiSim.

8.6 Steady vs. transient, convergence, residuals

The solver runs steady-state or transient (with time step dt). Convergence is monitored through residuals plotted live with ImPlot. Residuals can be computed raw or RMS, with L_1 , L_2 , or L_∞ norms and optional scaling by N or $\sqrt{N}$ (`ResidualType`, `ResidualNormType`, `ResidualScalingType`). Plots are available for u , v , p , continuity, temperature, and concentration.

8.7 GPU acceleration and device data

At solve time the host `FVMesh` is packed and uploaded to the device as `FVMeshDevice`, which stores cells and faces in structure-of-arrays form with CSR-style face connectivity (`faceStart`, `faceIDs`). Per-field boundary data is uploaded per group (`BoundarySolverDevice`). Kernels assemble the coefficient matrices and run the iterative solves; CUDA streams and events coordinate and time the work.

Table 2: Boundary types and their typical field treatment.

Type	Treatment
Wall	No-slip velocity (Dirichlet $u = v = 0$); zero-gradient pressure.
Velocity inlet	Prescribed velocity (Dirichlet); zero-gradient pressure.
Pressure outlet	Prescribed pressure (Dirichlet); zero-gradient velocity.
Symmetry	Zero normal velocity and zero normal gradient (centreline / planes of symmetry).



Figure 7: Running the solver with live residual monitoring.

9 Post-processing and Results

9.1 3D results view

The Results tab revolves the 2D solution into a 3D model that you can orbit, pan, and zoom with an arcball camera (Section 10). The field is coloured by the active colormap, the geometry can be sliced, and an outline is drawn so the cylinder's silhouette and cap edges remain distinct.

9.2 Data picker (3D)

Clicking the 3D surface returns the field value at that location. The click ray is intersected with the cylinder (end caps plus outer surface), and the nearest valid intersection is used. The value is then obtained by bilinear interpolation of the 2D solution; because the solution is axisymmetric, only the axial and radial directions are needed. Care is taken near boundaries because field values are stored at cell centres.

9.3 2D inspector and per-cell mesh data

The 2D inspector renders the solution directly over the finite-volume mesh (each cell coloured by its value, with flat or interpolated shading) using the same `ImDrawList`/2D-camera approach as the sketch and mesh views. A value probe shows the field value under the cursor. The mesh-data

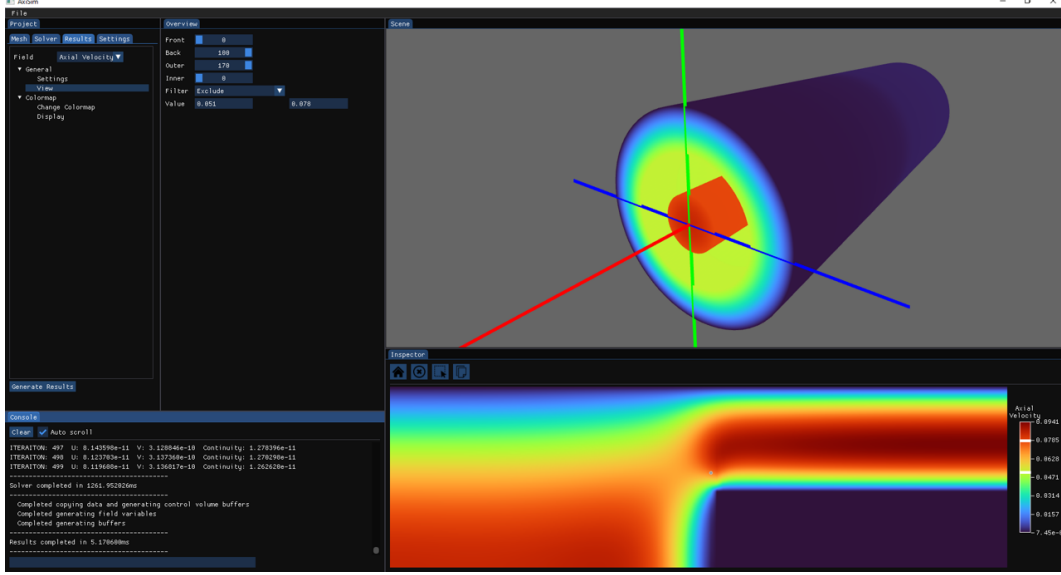


Figure 8: Results view: a revolved field with the 2D cross-section inspector.

inspect tool lets you click a cell to read its geometric and quality data—centroid, area/volume, face count, per-face neighbour, edge length, and non-orthogonality, plus, where available, per-cell continuity and face mass fluxes.

9.4 Colorbar and colormaps

A colorbar accompanies the field views. Colormaps (Turbo, Parula, HSV, Gray) are stored as 256-entry lookup tables compiled into the application; the colorbar and field views sample the active colormap and clamp to each field’s min/max. The colormap can also be changed from the console (Section 11).

10 Interactive Features and Camera

10.1 3D arcball camera

The 3D view uses an arcball camera built on quaternions (avoiding gimbal lock). Holding the middle mouse button and dragging rotates the camera along a spherical path about the target; the rotation axis and angle come from mapping the previous and current cursor positions $\hat{\mathbf{V}}_a, \hat{\mathbf{V}}_b$ onto a sphere:

$$\text{Axis} = \frac{\hat{\mathbf{V}}_a \times \hat{\mathbf{V}}_b}{|\hat{\mathbf{V}}_a \times \hat{\mathbf{V}}_b|}, \quad \theta = \arccos(\hat{\mathbf{V}}_a \cdot \hat{\mathbf{V}}_b). \quad (5)$$

Clicking an axis snaps the camera to an axis-aligned view by **slerp**-interpolating from the current to a target orientation.

10.2 2D camera

The sketch, mesh, and 2D-inspector views share a 2D camera with wheel zoom and middle-button pan, plus world/screen coordinate conversion used for picking and overlays.

10.3 Mouse picking

Selection in 3D uses ray casting against object bounding boxes (slab method); each box has a unique ID so the program can dispatch the correct action on a hit. In 2D, cell and boundary picking use point-in-triangle and point-to-segment tests in world space.

11 Console

The console accepts text commands composed of an *action*, an *object*, and an optional *value*. For example, `run solver` runs the solver, and `set colormap Turbo` changes the colormap. Tables 3–5 list the recognised keywords.

Table 3: Actions.

<code>run</code>	run an operation
<code>set</code>	set a setting
<code>clear/clr</code>	clear the line

Table 4: Objects.

<code>mesh</code>	mesh
<code>solver</code>	solver
<code>results</code>	results
<code>colormap/cmap</code>	colormap

Table 5: Values.

Turbo, Parula, HSV, Gray (colormaps, case- sensitive)
--

12 Data Structures Reference

Table 6 summarises the central data structures. The host representation (`FVMesh`, Section 7.3) is the source of truth; a packed structure-of-arrays form is uploaded to the GPU for solving.

Table 6: Key data structures.

Structure	Purpose
<code>GridConfig</code>	Structured grid: n_r, n_z, L, R , face/centre coordinates, grading, <code>activeCell</code> , <code>obstacleIndices</code> , and device pointers.
<code>FluidPropertyConfig</code>	Fluid and transport properties: density ρ , viscosity μ , c_p , k , diffusivities, and reaction parameters.
<code>Triangle</code>	Three vertex indices; one triangle = one unstructured FV cell.
<code>FVCell</code> / <code>FVFace</code> / <code>FVMesh</code>	Host finite-volume mesh: cell geometry, face owner/neighbour/normal/area, connectivity.
<code>FVMeshDevice</code>	Device (CUDA) mesh in structure-of-arrays form with CSR face connectivity.
<code>ConfigSolver</code> / <code>ConfigSimple</code>	Solver configuration: schemes, transient/ dt , tolerances, corrector passes.
<code>VariablesSimple</code>	Device field arrays for SIMPLE (u, v, p, p', T , gradients, mass flux) and relaxation factors.
<code>Coefficients</code>	Per-field linear-system coefficients ($A_E, A_W, A_N, A_S, A_C, b$, residuals).
<code>BoundarySegmentGroup</code>	A named group of boundary segments carrying a boundary type and per-field values.

13 Save and Load

Project state is saved to and loaded from binary (`.bin`) files to keep file size small. Geometry, mesh, solver settings, and boundary groups are persisted per tab and can be loaded manually or automatically at launch, which speeds up iterating on test cases. The binary format favours compactness over editability and may change between versions.

14 Limitations

- **Not verified or validated.** Results have not been checked against analytical solutions, reference solvers, or experiments (Section 2).
- **Alpha software.** Expect bugs, incomplete features, and breaking changes between versions; saved files may not be forward/backward compatible.
- **Physics scope.** Incompressible, axisymmetric (no swirl/3D), laminar flow only—no turbulence modelling. Scalar transport (energy, concentration) is passive/one-way unless otherwise configured.
- **Solver scope.** SIMPLE is the implemented pressure–velocity coupling; other schemes (SIMPLER, SIMPLEC, PISO) are not yet available.
- **Non-orthogonal correction.** Off by default; the deferred cross term can destabilise on skewed or near-axis cells, so it is opt-in and limited.
- **Platform.** Windows x64 only, and it requires a CUDA-capable NVIDIA GPU (compute capability ≥ 7.5); it will not run on AMD/Intel GPUs or CPU-only machines. Single-GPU only.
- **Scale / memory.** Large meshes are bounded by GPU memory; very large cases may not fit or may be slow.
- **Robustness.** Convergence is sensitive to mesh quality, relaxation factors, and boundary conditions; poor inputs can diverge.

Appendix A: Default Parameters

Table 7: Selected default grid and fluid parameters (subject to change).

Parameter	Symbol / field	Default
Axial cells	nz	100
Radial cells	nr	170
Domain length	L	0.01 m
Domain radius	R	0.0017 m
Density	rho	998 kg m ⁻³ (water)
Dynamic viscosity	mu	1.0518×10^{-3} Pa s
Specific heat	cp	4180 J kg ⁻¹ K ⁻¹
Thermal conduct.	k	0.6 W m ⁻¹ K ⁻¹
Concentration diff.	D	3.0277×10^{-9} m ² s ⁻¹
Momentum relax.	—	0.7
Pressure relax.	—	0.3

References

- [1] S. V. Patankar, *Numerical Heat Transfer and Fluid Flow*, Hemisphere, 1980.
- [2] H. K. Versteeg and W. Malalasekera, *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*, 2nd ed., Pearson, 2007.
- [3] C. Geuzaine and J.-F. Remacle, “Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities,” *Int. J. Numer. Methods Eng.*, 79(11), 2009.
- [4] O. Cornut, *Dear ImGui*, <https://github.com/ocornut/imgui>.
- [5] LDraw.org Wiki, “Conditional Line,” https://wiki.ldraw.org/wiki/Conditional_Line, 2018.
- [6] LDraw.org, “LDraw File Format Specification,” <https://www.ldraw.org/article/218.html>.